System Setup Guide

ESP32 Balanced Robot Swarm & Ground Control Station

1. Hardware Requirements

- **Microcontrollers:** 2× ESP32 Development Boards (One for the Robot, one for the Controller).
- **Robot Sensors:** 2× Adafruit BNO055 IMUs, 1× INA219 Voltage Sensor.
- **Robot Motors:** 1× Standard PWM motor (Main Wheel), Waveshare ST3215 Serial Bus Servos (Pendulums).
- **Input:** Any standard USB or Bluetooth Gamepad (Xbox, PlayStation, or generic).

2. Firmware Compilation & Flashing

The codebase is designed so that both the Robot and the Controller share a root directory, but compile completely different subsets of the code based on preprocessor macros.

Step 2.1: Obtain MAC Addresses

Before configuring the devices, you must know the ESP-NOW MAC addresses of both your Robot(s) and your Controller. Flash a basic sketch to both ESP32s to print their Wi-Fi Station MAC addresses to the serial monitor. Note these down.

Step 2.2: Configure the Controller

1. Open `controller/controller.cpp`.
2. Locate the `robotMacs` array and replace the default MAC addresses with the one(s) you obtained for your Robot(s) in Step 2.1.
3. Open your build system configuration (e.g., `platformio.ini`) or your main sketch file and ensure the `ROLE_CONTROLLER` macro is defined.
4. Compile and flash the firmware to your **Controller ESP32**.

```
// In controller.cpp
uint8_t robotMacs[NUM_ROBOTS][6] = {
    {0xAA, 0xBB, 0xCC, 0xDD, 0xEE, 0xFF}, // Replace with your Robot's MAC
};
```

Step 2.3: Configure the Robot

1. Open `robotconfig.cpp` (or `robot_config.h` depending on your file structure version) and **set the Controller MAC address**. This ensures the robot knows which specific controller it should listen to and trust.
2. Open your build system configuration and ensure the `ROLE_ROBOT` macro is defined.
3. Ensure you have the required Arduino libraries installed: `Adafruit_BNO055`, `Adafruit_INA219`, `ESP32Servo`.
4. Compile and flash the firmware to your **Robot ESP32**.

> **Important: First Boot Behavior**
>
> Upon the first successful boot, the robot will enter a
> **Calibration Required**
> state (indicated by a rapid LED blink pattern). The motors will remain disabled until an IMU calibration sequence is successfully completed via the Python GUI.

3. Python Ground Control Station (GCS) Setup

Step 3.1: Install Dependencies

The Ground Control Station requires Python 3.8 or newer. Open your terminal and install the required packages:

```
pip install pyserial pygame matplotlib
```

Step 3.2: Launch the Dashboard

Navigate to the `Code/` directory containing the Python scripts and execute the main entry point:

```
python main.py
```

4. System Pairing & Calibration

Step 4.1: Connect to the Controller

In the Python GUI, navigate to the **Configuration** tab. Select the COM port corresponding to your connected **Controller ESP32** (not the Robot). Leave the baud rate at `115200` and click **Connect**. The System Log should indicate a successful connection.

Step 4.2: Bind the Gamepad

1. Plug in or connect your gamepad.
2. Navigate to the **Game Controller** tab. Your controller should automatically appear in the drop-down menu.
3. In the *Active Robot Assignments* section, ensure your Robot ID is selected.
4. Use the **Bind Input** buttons to map your joystick axes and buttons to the corresponding actions (Forward/Back, Strafe, E-STOP, Arm).
5. Click **Apply Settings to Robot**.

Step 4.3: Initial IMU Calibration Sequence

By default, the robot boots in a locked state requiring calibration.

1. Navigate to the **Live View** tab. You should see your robot listed with a purple "CALIBRATION REQUIRED" banner.
2. Click the **Calibrate Full** button.
3. Follow the on-screen confirmation prompts. You will be asked to hold the robot perfectly upright, approve the state, and then tilt the robot 90 degrees forward and approve again.
4. Once complete, the robot's state will change to **DISARMED (Neutral Stream)**. The calibration offsets are now saved to the Robot's non-volatile storage (NVS) and will persist across reboots.

> **Caution: Developer Mode (Bypassing Calibration)**
>
> If you are actively developing and don't want to calibrate the robot every time you reset the NVS memory, you can circumvent the calibration requirement. Navigate to the **Parameter Tuning** tab, fetch the `dev_mode` setting under the "System & Misc" group, and set it to `1.0`.
>
> *Warning: This should **only** be done during development/bench testing. Bypassing calibration on a physical robot on the ground can lead to highly unstable and unpredictable balancing behavior.*

Step 4.4: Arm and Drive

With the robot powered, calibrated (or in dev mode), and on the ground, press the **Arm** button on your mapped gamepad. The robot will transition to **ARMED & READY** and its PID loops will engage to balance the chassis. Use the joysticks to drive!